

LAPORAN PRAKTIKUM

Workshop Web Dasar

Javascript DOM

*(Implementasi Aplikasi Todo List Menggunakan Javascript
DOM)*



DISUSUN OLEH:

FAQROEL RAZI

2025573010033

TEKNIK INFORMATIKA

POLITEKNIK NEGERI LHOKSEUMAWE

2026

KATA PENGANTAR

Puji syukur kehadiran Allah SWT atas segala rahmat dan karunia-Nya sehingga laporan praktikum yang berjudul "**JavaScript DOM**" ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk dokumentasi dan pertanggungjawaban atas kegiatan praktikum yang telah dilaksanakan.

Praktikum ini bertujuan untuk mempelajari konsep **Document Object Model (DOM)** pada JavaScript serta penerapannya dalam pembuatan website yang bersifat dinamis dan interaktif. Melalui praktikum ini, mahasiswa dapat memahami bagaimana JavaScript digunakan untuk mengakses, memodifikasi, dan mengelola elemen-elemen pada halaman web secara langsung.

Penyusunan laporan ini tidak terlepas dari bantuan berbagai pihak. Oleh karena itu, penulis mengucapkan terima kasih kepada **Bapak M. Reza Zulman, M.Sc.** selaku dosen pengampu yang telah memberikan arahan dan bimbingan selama kegiatan praktikum berlangsung.

Penulis menyadari bahwa laporan ini masih memiliki berbagai kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan guna perbaikan di masa yang akan datang. Semoga laporan ini dapat memberikan manfaat bagi pembaca dan menjadi sumber pembelajaran dalam memahami JavaScript DOM.

Lhokseumawe, 5 Juni 2026

Penulis

DAFTAR ISI

LAPORAN PRAKTIKUM.....	i
KATA PENGANTAR.....	ii
DAFTAR ISI.....	iii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Identifikasi Masalah	1
1.3 Rumusan Masalah	2
BAB II LANDASAN TEORI	3
2.1 Document Object Model (DOM)	3
2.2 Memilih Elemen (Selecting Elements)	3
2.3 Manipulasi Elemen DOM	3
2.4 Manipulasi Struktur DOM	4
2.5 addEventListener.....	4
BAB III METODOLOGI PRAKTIKUM.....	6
3.1 Alat dan Bahan	6
3.2 Prosedur Praktikum.....	6
3.2.1 Setup project.....	6
3.2.2 Mengisi konten didalam file index.html	7
3.2.3 Mengatur tampilan konten html pada style.css	8
3.2.4 Membuat dynamic website dengan menggunakan DOM pada file script.js	11
BAB IV HASIL DAN PEMBAHASAN.....	16

4.1 Hasil Praktikum.....	16
4.2 Pembahasan.....	16
BAB V KESIMPULAN DAN SARAN	18
5.1. Kesimpulan	18
5.2. Saran.....	18
DAFTAR PUSTAKA.....	19

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi web modern menuntut website untuk tidak hanya menampilkan informasi secara statis, tetapi juga mampu memberikan interaksi yang responsif kepada pengguna. Untuk memenuhi kebutuhan tersebut, diperlukan bahasa pemrograman yang dapat mengelola dan memanipulasi tampilan halaman web secara dinamis. JavaScript menjadi salah satu teknologi utama yang digunakan untuk tujuan tersebut.

Salah satu konsep penting dalam JavaScript adalah Document Object Model (DOM). DOM memungkinkan JavaScript mengakses struktur dokumen HTML sebagai sekumpulan objek yang dapat dimanipulasi. Dengan memanfaatkan DOM, pengembang dapat mengubah isi, atribut, gaya, maupun struktur halaman web tanpa harus memuat ulang halaman.

Pemahaman terhadap JavaScript DOM menjadi dasar penting dalam pengembangan aplikasi web modern. Oleh karena itu, praktikum ini dilaksanakan untuk memberikan pengalaman langsung kepada mahasiswa dalam membangun website dinamis menggunakan berbagai teknik manipulasi DOM sehingga mampu meningkatkan pemahaman terhadap konsep interaktivitas pada web.

1.2 Identifikasi Masalah

Berdasarkan latar belakang tersebut, dapat diidentifikasi beberapa permasalahan sebagai berikut:

1. Kurangnya pemahaman mengenai konsep Document Object Model (DOM).
2. Kesulitan dalam mengakses dan memilih elemen HTML menggunakan JavaScript.

3. Kurangnya pemahaman tentang manipulasi elemen dan struktur halaman web secara dinamis.
4. Belum memahami penggunaan event untuk menciptakan interaksi pengguna pada website.
5. Perlunya penerapan JavaScript DOM dalam pembuatan website dinamis.

1.3 Rumusan Masalah

Adapun rumusan masalah pada praktikum ini adalah sebagai berikut:

1. Apa yang dimaksud dengan Document Object Model (DOM)?
2. Bagaimana cara memilih elemen HTML menggunakan JavaScript?
3. Bagaimana cara melakukan manipulasi elemen dan struktur DOM?
4. Bagaimana penggunaan event listener dalam JavaScript?
5. Bagaimana penerapan JavaScript DOM dalam pembuatan website dinamis?

BAB II

LANDASAN TEORI

2.1 Document Object Model (DOM)

Document Object Model (DOM) merupakan representasi terstruktur dari dokumen HTML atau XML dalam bentuk objek dan node yang dapat diakses serta dimanipulasi menggunakan bahasa pemrograman seperti JavaScript. DOM berfungsi sebagai penghubung antara dokumen web dengan script sehingga memungkinkan perubahan isi halaman secara dinamis tanpa perlu memuat ulang halaman.

2.2 Memilih Elemen (Selecting Elements)

Sebelum melakukan manipulasi terhadap halaman web, JavaScript harus terlebih dahulu memilih elemen yang akan diakses. Beberapa metode yang umum digunakan antara lain:

- `getElementById()`
- `getElementsByClassName()`
- `getElementsByTagName()`
- `querySelector()`
- `querySelectorAll()`

Metode tersebut memungkinkan pengembang memperoleh referensi terhadap elemen HTML tertentu sehingga dapat dimodifikasi sesuai kebutuhan.

2.3 Manipulasi Elemen DOM

Manipulasi elemen DOM dilakukan untuk mengubah isi maupun atribut suatu elemen HTML. Beberapa operasi yang sering digunakan meliputi:

- Mengubah teks menggunakan `textContent`
- Mengubah isi HTML menggunakan `innerHTML`

- Mengubah atribut menggunakan `setAttribute()`
- Mengubah gaya menggunakan properti `style`
- Menambah atau menghapus class menggunakan `classList`

Melalui manipulasi ini, tampilan website dapat berubah secara dinamis berdasarkan tindakan pengguna.

2.4 Manipulasi Struktur DOM

Selain mengubah isi elemen, JavaScript juga dapat mengubah struktur halaman web dengan menambahkan, memindahkan, atau menghapus elemen tertentu. Beberapa metode yang digunakan antara lain:

- `createElement()`
- `appendChild()`
- `prepend()`
- `remove()`
- `replaceChild()`

Manipulasi struktur DOM sangat berguna dalam pembuatan daftar data, menu dinamis, maupun berbagai komponen interaktif lainnya.

2.5 `addEventListener`

`addEventListener()` merupakan metode modern yang digunakan untuk menangani berbagai peristiwa (event) pada halaman web. Event dapat berupa klik tombol, pergerakan mouse, input pengguna, maupun interaksi lainnya.

Contoh penggunaan:

```
button.addEventListener("click", function() {
    alert("Tombol diklik");
});
```


Keunggulan metode ini adalah mampu menambahkan lebih dari satu event handler pada elemen yang sama serta memisahkan struktur HTML dari logika JavaScript sehingga kode menjadi lebih terorganisir dan mudah dipelajari.

BAB III

METODOLOGI PRAKTIKUM

3.1 Alat dan Bahan

Berdasarkan praktikum yang dilakukan, alat dan bahan yang diperlukan dalam implementasi website todo list adalah:

- Komputer atau Laptop
- IDE Visual Studio Code
- Git
- GitHub
- Browser

3.2 Prosedur Praktikum

3.2.1 Setup project

Membuat folder tugas untuk menyimpan project website todo list:

1. Buka git bash dan navigasikan ke folder repository GitHub yang telah di clone.
2. Buat direktori modul-6
3. Masuk ke direktori modul-6 lalu
4. Buat direktori tugas
5. Masuk ke direktori tugas lalu
6. Buat file index.html, style.css, script.js

```

ASUS@LAPTOP-F10SDSTJ MINGW64 /d/2025573010033_Workshop-Web-Dasar (main)
• $ mkdir modul_6

ASUS@LAPTOP-F10SDSTJ MINGW64 /d/2025573010033_Workshop-Web-Dasar (main)
• $ cd modul_6/

ASUS@LAPTOP-F10SDSTJ MINGW64 /d/2025573010033_Workshop-Web-Dasar/modul_6 (main)
• $ mkdir tugas

ASUS@LAPTOP-F10SDSTJ MINGW64 /d/2025573010033_Workshop-Web-Dasar/modul_6 (main)
• $ cd tugas

ASUS@LAPTOP-F10SDSTJ MINGW64 /d/2025573010033_Workshop-Web-Dasar/modul_6/tugas (main)
• $ touch index.html style.css script.js

ASUS@LAPTOP-F10SDSTJ MINGW64 /d/2025573010033_Workshop-Web-Dasar/modul_6/tugas (main)
• $ 

```

3.2.2 Mengisi konten didalam file index.html

Bukalah file index.html yang telah dibuat, dan isilah dengan konten yang dikerjakan dalam praktikum:

```

modul_6 > tugas > index.html
1 <!DOCTYPE html>
2 <html lang="id">
3 <head>
4   <meta charset="UTF-8" />
5   <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6   <title>Daftar Tugas</title>
7   <link rel="stylesheet" href="style.css" />
8 </head>
9 <body>
10   <main class="app-shell">
11     <header class="app-header">
12       <div>
13         <h1>Todo List Interaktif</h1>
14         <p class="lead">Tambah tugas, pilih prioritas, tandai selesai, filter, dan simpan otomatis di browser.</p>
15       </div>
16       <div class="counter-box">
17         <span id="remainingCount">0</span>
18         <p>tugas terminas</p>
19       </div>
20     </header>
21     <section class="task-form-card">
22       <form id="taskForm" novalidate>
23         <label for="taskInput">Tambah tugas baru</label>
24         <div class="input-group">
25           <input id="taskInput" name="task" type="text" placeholder="Contoh: Belajar DOM" maxlength="100" required />
26           <select id="priorityInput" name="priority" aria-label="Pilih prioritas tugas">
27             <option value="sedang" selected>Sedang</option>
28             <option value="rendah">Rendah</option>
29             <option value="tinggi">Tinggi</option>
30           </select>
31           <button type="submit">Tambah</button>
32         </div>
33         <p class="error-message" id="inputError" aria-live="polite"></p>
34       </form>
35     </section>
36     <section class="task-controls">
37       <div class="filters" role="group" aria-label="Filter tugas">
38         <button type="button" class="filter-button active" data-filter="semua">Semua</button>
39         <button type="button" class="filter-button" data-filter="aktif">Aktif</button>
40         <button type="button" class="filter-button" data-filter="selesai">Selesai</button>
41       </div>
42       <button type="button" class="secondary-button" id="clearCompleted">Hapus Semua Selesai</button>
43     </section>
44     <section class="task-list-card">
45       <ul id="taskList" class="task-list" aria-live="polite"></ul>
46     </section>
47   </main>
48   <script src="script.js"></script>
49 </body>
50 </html>
51

```

3.2.3 Mengatur tampilan konten html pada style.css

Bukalah file style.css yang telah dibuat, dan aturlah tampilan konten yang telah diisi pada file index.html:

```
modul6>tugas> style.css> root
1  :root {
2    color-scheme: light;
3    font-family: Inter, ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif;
4    background: #eef2ff;
5    color: #0f172a;
6  }
7
8  * {
9    box-sizing: border-box;
10 }
11
12 html, body {
13   margin: 0;
14   min-height: 100%;
15 }
16
17 body {
18   background: linear-gradient(180deg, #eef2ff 0%, #f8f9fc 100%);
19 }
20
21 button, input, select {
22   font: inherit;
23 }
24
25 .app-shell {
26   width: min(1100px, calc(100% - 32px));
27   margin: 0 auto;
28   padding: 32px 0 64px;
29   display: grid;
30   gap: 24px;
31 }
32
33 .app-header {
34   display: grid;
35   gap: 18px;
36   padding: 28px 28px 24px;
37   border-radius: 28px;
38   background: white;
39   box-shadow: 0 20px 60px rgba(15, 23, 42, 0.08);
40 }
41
42 .eyebrow {
43   margin: 0;
44   color: #2563eb;
45   font-weight: 700;
46   letter-spacing: 0.08em;
47   text-transform: uppercase;
48   font-size: 0.78rem;
49 }
50
51 h1 {
52   margin: 0;
53   font-size: clamp(2rem, 2.8vw, 3rem);
54 }
55
56 .lead {
57   margin: 0;
58   color: #475569;
59   max-width: 760px;
60   line-height: 1.75;
61 }
62
63 .counter-box {
64   align-self: center;
65   justify-self: end;
66   background: #eef2ff;
67   border: 1px solid #dbeafe;
68   border-radius: 24px;
69   padding: 16px 20px;
70   text-align: center;
71   min-width: 140px;
72 }
73
74 .counter-box span {
75   display: block;
76   font-size: 2rem;
77   font-weight: 800;
78 }
79
80 .counter-box p {
81   margin: 4px 0 0;
82   color: #475569;
83 }
84
```

```

modul-6 > tugas > style.css > root
85 .task-form-card,
86 .task-list-card,
87 .task-controls {
88   background: white;
89   border-radius: 28px;
90   box-shadow: 0 28px 60px rgba(15, 23, 42, 0.08);
91   padding: 24px;
92 }
93
94 .task-form-card label {
95   display: block;
96   margin-bottom: 10px;
97   font-weight: 700;
98 }
99
100 .input-group {
101   display: grid;
102   grid-template-columns: 1fr 160px 140px;
103   gap: 12px;
104 }
105
106 .input-group input,
107 .input-group select {
108   width: 100%;
109   border-radius: 16px;
110   border: 1px solid #cbd5e1;
111   padding: 14px 16px;
112   outline: none;
113   transition: border-color 0.2s ease, box-shadow 0.2s ease;
114 }
115
116 .input-group input:focus,
117 .input-group select:focus {
118   border-color: #2563eb;
119   box-shadow: 0 0 4px rgba(59, 130, 246, 0.12);
120 }
121

```

```

modul-6 > tugas > style.css > root
122 .input-group button {
123   border: none;
124   border-radius: 16px;
125   background: #2563eb;
126   color: white;
127   padding: 0 18px;
128   min-width: 120px;
129   cursor: pointer;
130   font-weight: 700;
131   transition: transform 0.15s ease, background 0.15s ease;
132 }
133
134 .input-group button:hover {
135   transform: translateY(-1px);
136   background: #1d4ed8;
137 }
138
139 .error-message {
140   margin: 12px 0 0;
141   color: #dc2626;
142   min-height: 1.2rem;
143 }
144
145 .task-controls {
146   display: flex;
147   flex-wrap: wrap;
148   justify-content: space-between;
149   gap: 16px;
150 }
151
152 .filters {
153   display: flex;
154   gap: 12px;
155   flex-wrap: wrap;
156 }
157
158 .filter-button

```

```

158 .filter-button,
159 .secondary-button {
160   border: 1px solid #cbd5e1;
161   border-radius: 14px;
162   background: #white;
163   color: #0f172a;
164   padding: 12px 18px;
165   cursor: pointer;
166   transition: background 0.15s ease, border-color 0.15s ease, color 0.15s ease;
167 }
168
169 .filter-button.active,
170 .filter-button:hover,
171 .secondary-button:hover {
172   background: #eaf2ff;
173   border-color: #93c5fd;
174 }
175
176 .secondary-button {
177   min-width: 220px;
178 }
179
180 .task-list {
181   margin: 0;
182   padding: 0;
183   list-style: none;
184   display: grid;
185   gap: 18px;
186 }
187
188 .task-item {
189   display: grid;
190   grid-template-columns: auto 1fr auto;
191   gap: 16px;
192   align-items: center;
193   padding: 18px 22px;
194   border-radius: 22px;
195   border: 1px solid #e2e8f0;
196   background: #f8fafc;
197 }
198
199 .task-item.dragging {
200   opacity: 0.5;
201 }
202

```

```

modul-6 > tugas > style.css > % root
203 .task-item[data-priority="tinggi"] {
204   border-left: 6px solid #dc2626;
205 }
206
207 .task-item[data-priority="sedang"] {
208   border-left: 6px solid #f59e0b;
209 }
210
211 .task-item[data-priority="rendah"] {
212   border-left: 6px solid #10b981;
213 }
214
215 .task-main {
216   display: grid;
217   gap: 10px;
218 }
219
220 .task-main label {
221   display: flex;
222   align-items: center;
223   gap: 14px;
224   cursor: pointer;
225 }
226
227 .task-main input[type="checkbox"] {
228   width: 18px;
229   height: 18px;
230 }
231
232 .task-text {
233   margin: 0;
234   font-weight: 600;
235 }
236
237 .task-text.completed {
238   text-decoration: line-through;
239   color: #64748b;
240 }
241
242 .task-meta {
243   display: flex;
244   align-items: center;
245   gap: 10px;
246   color: #475569;
247   font-size: 0.95rem;
248 }

```

```

modul6 > tugas > style.css > root
250 .priority-pill {
251   display: inline-flex;
252   align-items: center;
253   gap: 6px;
254   border-radius: 999px;
255   padding: 6px 10px;
256   background: rgba(59, 130, 246, 0.1);
257   color: #1d4ed8;
258 }
259
260 .task-actions {
261   display: flex;
262   gap: 10px;
263   align-items: center;
264 }
265
266 .task-actions button {
267   border: none;
268   background: transparent;
269   color: #475569;
270   cursor: pointer;
271   font-weight: 700;
272   padding: 8px 10px;
273   border-radius: 14px;
274   transition: background 0.15s ease, color 0.15s ease;
275 }
276
277 .task-actions button:hover {
278   background: rgba(148, 163, 184, 0.15);
279 }
280
281 .edit-input {
282   width: 100%;
283   border: 1px solid #cbd5e1;
284   border-radius: 14px;
285   padding: 10px 14px;
286 }
287
288 .empty-state {
289   padding: 34px;
290   text-align: center;
291   color: #64748b;
292   border-radius: 24px;
293   border: 1px dashed #cbd5e1;
294 }
295

```

```

296 @media (max-width: 760px) {
297   .input-group {
298     grid-template-columns: 1fr;
299   }
300
301   .task-controls {
302     flex-direction: column;
303   }
304
305   .task-item {
306     grid-template-columns: auto 1fr;
307   }
308
309   .task-actions {
310     justify-self: end;
311   }
312 }
313

```

3.2.4 Membuat dynamic website dengan menggunakan DOM pada file script.js

Buka file script.js yang telah dibuat, kemudian tambahkan kode JavaScript untuk mengatur dan memanipulasi elemen pada halaman web berdasarkan struktur yang ada pada index.html dan tampilan yang telah ditentukan pada style.css :

```

module-6 > tugas > scripts > validateTask
1  const STORAGE_KEY = 'tugasModul6';
2
3  const taskForm = document.getElementById('taskForm');
4  const taskInput = document.getElementById('taskInput');
5  const priorityInput = document.getElementById('priorityInput');
6  const inputError = document.getElementById('inputError');
7  const taskList = document.getElementById('taskList');
8  const remainingCount = document.getElementById('remainingCount');
9  const filterButtons = document.querySelectorAll('.filter-button');
10 const clearCompletedButton = document.getElementById('clearCompleted');
11
12 let tasks = [];
13 let activeFilter = 'semua';
14 let dragSourceId = null;
15
16 Windsurf: Refactor | Explain | Generate JSDoc
17 function saveTasks() {
18   localStorage.setItem(STORAGE_KEY, JSON.stringify(tasks));
19 }
20
21 Windsurf: Refactor | Explain | Generate JSDoc
22 function loadTasks() {
23   const stored = localStorage.getItem(STORAGE_KEY);
24   tasks = stored ? JSON.parse(stored) : [];
25 }
26
27 Windsurf: Refactor | Explain | Generate JSDoc
28 function validateTask(text) {
29   const trimmed = text.trim();
30   if (!trimmed) {
31     return 'Tugas tidak boleh kosong.';
32   }
33   if (trimmed.length < 3) {
34     return 'Tugas minimal 3 karakter.';
35   }
36   if (trimmed.length > 100) {
37     return 'Tugas maksimal 100 karakter.';
38   }
39   return '';
40 }
41
42 Windsurf: Refactor | Explain | Generate JSDoc
43 function getRemainingCount() {
44   return tasks.filter(task => !task.completed).length;
45 }
46
47 Windsurf: Refactor | Explain | Generate JSDoc
48
49 module-6 > tugas > scripts > validateTask
50
51 Windsurf: Refactor | Explain | Generate JSDoc
52 function filterTasks() {
53   if (activeFilter === 'aktif') {
54     return tasks.filter(task => !task.completed);
55   }
56   if (activeFilter === 'selesai') {
57     return tasks.filter(task => task.completed);
58   }
59   return tasks;
60 }
61
62 Windsurf: Refactor | Explain | Generate JSDoc
63 function sortPriorityLabel(priority) {
64   return priority === 'tinggi' ? 'tinggi' : priority === 'rendah' ? 'Rendah' : 'Sedang';
65 }
66
67 Windsurf: Refactor | Explain | Generate JSDoc
68 function createTaskItem(task) {
69   const listItem = document.createElement('li');
70   listItem.className = 'task-item';
71   listItem.dataset.id = task.id;
72   listItem.dataset.priority = task.priority;
73   listItem.draggable = true;
74
75   const checkbox = document.createElement('input');
76   checkbox.type = 'checkbox';
77   checkbox.checked = task.completed;
78   checkbox.addEventListener('change', () => toggleTaskCompleted(task.id));
79
80   const label = document.createElement('label');
81   label.appendChild(checkbox);
82
83   const taskText = document.createElement('p');
84   taskText.className = 'task-text';
85   taskText.textContent = task.text;
86   if (task.completed) {
87     taskText.classList.add('completed');
88   }
89   taskText.addEventListener('dblclick', () => beginTaskEdit(task.id, taskText));
90   label.appendChild(taskText);
91 }

```



```

modul-6 > tugas > scriptjs > createTaskItem
82 const meta = document.createElement('div');
83 meta.className = 'task-meta';
84 meta.innerHTML = `
85   <span class="priority-pill">${sortPriorityLabel(task.priority)}</span>
86   <span>${task.completed ? 'Selesai' : 'Aktif'}</span>
87 `;
88
89 const taskMain = document.createElement('div');
90 taskMain.className = 'task-main';
91 taskMain.appendChild(label);
92 taskMain.appendChild(meta);
93
94 const actions = document.createElement('div');
95 actions.className = 'task-actions';
96
97 const prioritySelect = document.createElement('select');
98 prioritySelect.className = 'priority-select';
99 prioritySelect.innerHTML = `
100   <option value="rendah" ${task.priority === 'rendah' ? 'selected' : ''}>Rendah</option>
101   <option value="sedang" ${task.priority === 'sedang' ? 'selected' : ''}>Sedang</option>
102   <option value="tinggi" ${task.priority === 'tinggi' ? 'selected' : ''}>Tinggi</option>
103 `;
104 prioritySelect.addEventListener('change', () => updateTaskPriority(task.id, prioritySelect.value));
105
106 const deleteButton = document.createElement('button');
107 deleteButton.type = 'button';
108 deleteButton.textContent = 'Hapus';
109 deleteButton.addEventListener('click', () => deleteTask(task.id));
110
111 actions.appendChild(prioritySelect);
112 actions.appendChild(deleteButton);
113
114 listItem.appendChild(taskMain);
115 listItem.appendChild(actions);
116

```

```

modul-6 > tugas > scriptjs > createTaskItem
117 - listItem.addEventListener('dragstart', () => {
118 -   dragSourceId = task.id;
119 -   listItem.classList.add('dragging');
120 - });
121
122 - listItem.addEventListener('dragend', () => {
123 -   listItem.classList.remove('dragging');
124 -   dragSourceId = null;
125 - });
126
127 - listItem.addEventListener('dragover', event => {
128 -   event.preventDefault();
129 -   listItem.classList.add('drag-over');
130 - });
131
132 - listItem.addEventListener('dragleave', () => {
133 -   listItem.classList.remove('drag-over');
134 - });
135
136 - listItem.addEventListener('drop', event => {
137 -   event.preventDefault();
138 -   listItem.classList.remove('drag-over');
139 -   const targetId = Number(listItem.dataset.id);
140 -   if (dragSourceId !== null && dragSourceId !== targetId) {
141 -     moveTask(dragSourceId, targetId);
142 -   }
143 - });
144
145 - return listItem;
146
147

```

```

modul-6 > tugas > script.js > createTaskItem
Windsurf: Refactor | Explain | Generate JSDoc
148 function renderTasks() {
149   taskList.innerHTML = '';
150   const visibleTasks = filterTasks();
151   remainingCount.textContent = getRemainingCount();
152
153   if (visibleTasks.length === 0) {
154     const emptyMessage = document.createElement('div');
155     emptyMessage.className = 'empty-state';
156     emptyMessage.textContent = 'Belum ada tugas yang sesuai kriteria.';
157     taskList.appendChild(emptyMessage);
158     return;
159   }
160
161   visibleTasks.forEach(task => taskList.appendChild(createTaskItem(task)));
162 }
163
Windsurf: Refactor | Explain | Generate JSDoc
164 function addTask(text, priority) {
165   const errorMessage = validateTask(text);
166   if (errorMessage) {
167     inputError.textContent = errorMessage;
168     return;
169   }
170
171   inputError.textContent = '';
172   tasks.push({
173     id: Date.now(),
174     text: text.trim(),
175     completed: false,
176     priority,
177   });
178   saveTasks();
179   renderTasks();
180   taskForm.reset();
181   taskInput.focus();
182 }
183

```

```

modul-6 > tugas > script.js > ...
184 function toggleTaskCompleted(id) {
185   tasks = tasks.map(task => task.id === id ? { ...task, completed: !task.completed } : task);
186   saveTasks();
187   renderTasks();
188 }
189
Windsurf: Refactor | Explain | Generate JSDoc
190 function deleteTask(id) {
191   tasks = tasks.filter(task => task.id !== id);
192   saveTasks();
193   renderTasks();
194 }
195
Windsurf: Refactor | Explain | Generate JSDoc
196 function updateTaskPriority(id, priority) {
197   tasks = tasks.map(task => task.id === id ? { ...task, priority } : task);
198   saveTasks();
199   renderTasks();
200 }
201
Windsurf: Refactor | Explain | Generate JSDoc
202 function clearCompletedTasks() {
203   tasks = tasks.filter(task => !task.completed);
204   saveTasks();
205   renderTasks();
206 }
207
Windsurf: Refactor | Explain | Generate JSDoc
208 function setFilter(filter) {
209   activeFilter = filter;
210   filterButtons.forEach(button => {
211     button.classList.toggle('active', button.dataset.filter === filter);
212   });
213   renderTasks();
214 }
215
Windsurf: Refactor | Explain | Generate JSDoc

```

```

modul-6 > tugas > scripts > ...
216 function beginTaskEdit(id, taskTextElement) {
217   const task = tasks.find(item => item.id === id);
218   if (!task) return;
219
220   const input = document.createElement('input');
221   input.type = 'text';
222   input.value = task.text;
223   input.className = 'edit-input';
224   input.maxLength = 100;
225   input.addEventListener('blur', () => finishTaskEdit(id, input));
226   input.addEventListener('keydown', event => {
227     if (event.key === 'Enter') {
228       input.blur();
229     }
230     if (event.key === 'Escape') {
231       renderTasks();
232     }
233   });
234
235   taskTextElement.replaceWith(input);
236   input.focus();
237   input.select();
238 }
239
240 function finishTaskEdit(id, inputElement) {
241   const newText = inputElement.value.trim();
242   const errorMessage = validateTask(newText);
243   if (errorMessage) {
244     inputError.textContent = errorMessage;
245     inputElement.focus();
246     return;
247   }
248
249   const task = tasks.find(task => task.id === id);
250   task.text = newText;
251   saveTasks();
252   renderTasks();
253 }
254
255 function moveTask(sourceId, targetId) {
256   const sourceIndex = tasks.findIndex(task => task.id === sourceId);
257   const targetIndex = tasks.findIndex(task => task.id === targetId);
258   if (sourceIndex < 0 || targetIndex < 0) return;
259
260   const [movedTask] = tasks.splice(sourceIndex, 1);
261   tasks.splice(targetIndex, 0, movedTask);
262   saveTasks();
263   renderTasks();
264 }
265
266 function initEventListeners() {
267   taskForm.addEventListener('submit', event => {
268     event.preventDefault();
269     addTask(taskInput.value, priorityInput.value);
270   });
271
272   filterButtons.forEach(button => {
273     button.addEventListener('click', () => setFilter(button.dataset.filter));
274   });
275
276   clearCompletedButton.addEventListener('click', clearCompletedTasks);
277 }
278
279 function init() {
280   loadTasks();
281   initEventListeners();
282   renderTasks();
283   init();
284 }

```

```

modul-6 > tugas > scripts > ...
249   inputError.textContent = '';
250   tasks = tasks.map(task => task.id === id ? { ...task, text: newText } : task);
251   saveTasks();
252   renderTasks();
253 }
254
255 function moveTask(sourceId, targetId) {
256   const sourceIndex = tasks.findIndex(task => task.id === sourceId);
257   const targetIndex = tasks.findIndex(task => task.id === targetId);
258   if (sourceIndex < 0 || targetIndex < 0) return;
259
260   const [movedTask] = tasks.splice(sourceIndex, 1);
261   tasks.splice(targetIndex, 0, movedTask);
262   saveTasks();
263   renderTasks();
264 }
265
266 function initEventListeners() {
267   taskForm.addEventListener('submit', event => {
268     event.preventDefault();
269     addTask(taskInput.value, priorityInput.value);
270   });
271
272   filterButtons.forEach(button => {
273     button.addEventListener('click', () => setFilter(button.dataset.filter));
274   });
275
276   clearCompletedButton.addEventListener('click', clearCompletedTasks);
277 }
278
279 function init() {
280   loadTasks();
281   initEventListeners();
282   renderTasks();
283   init();
284 }

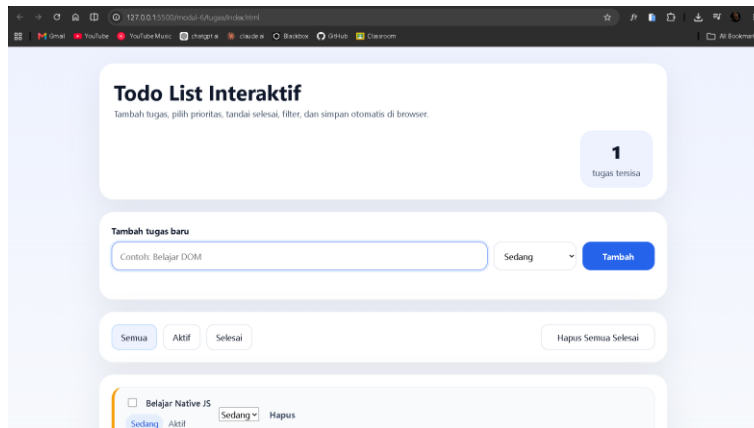
```

BAB IV

HASIL DAN PEMBAHASAN

4.1 Hasil Praktikum

Praktikum berhasil menghasilkan sebuah website dinamis yang memanfaatkan JavaScript DOM untuk mengelola interaksi pengguna. Fitur yang dibuat meliputi pemilihan elemen, perubahan isi halaman secara dinamis, manipulasi struktur DOM, serta penggunaan event listener untuk merespons tindakan pengguna.



4.2 Pembahasan

Pada praktikum ini, JavaScript DOM digunakan sebagai media untuk menghubungkan struktur HTML dengan logika pemrograman JavaScript. Berbagai elemen pada halaman web dapat dipilih menggunakan metode seleksi elemen, kemudian dimodifikasi sesuai kebutuhan aplikasi. Proses ini memungkinkan perubahan konten, atribut, maupun tampilan halaman secara langsung tanpa perlu melakukan refresh halaman sehingga meningkatkan pengalaman pengguna.

Selain manipulasi elemen, praktikum juga menerapkan penggunaan `addEventListener()` untuk menangani berbagai interaksi pengguna. Event yang diberikan memungkinkan website merespons tindakan seperti klik tombol atau

input data secara real-time. Penerapan konsep DOM dan event handling menunjukkan bagaimana JavaScript berperan penting dalam membangun website yang interaktif, dinamis, dan lebih responsif dibandingkan website statis.

BAB V

PENUTUP

5.1. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa JavaScript DOM merupakan teknologi yang memungkinkan pengembang mengakses dan memanipulasi elemen HTML secara dinamis. Melalui DOM, perubahan konten, atribut, gaya, dan struktur halaman web dapat dilakukan secara langsung menggunakan JavaScript.

Selain itu, penggunaan event listener memungkinkan website memberikan respons terhadap berbagai tindakan pengguna. Dengan memahami konsep DOM dan event handling, mahasiswa dapat membangun website yang lebih interaktif dan sesuai dengan kebutuhan pengembangan web modern.

5.2. Saran

- Mahasiswa disarankan untuk memperbanyak latihan manipulasi DOM pada berbagai jenis proyek web.
- Perlu mempelajari konsep event handling yang lebih kompleks untuk meningkatkan interaktivitas website.
- Praktikum selanjutnya dapat dikembangkan dengan mengintegrasikan DOM dengan API atau framework JavaScript modern.
- Dokumentasi kode sebaiknya dibuat dengan baik agar memudahkan proses pengembangan dan pemeliharaan aplikasi.

DAFTAR PUSTAKA

- Duckett, J. (2014). *JavaScript & jQuery: Interactive Front-End Web Development*. John Wiley & Sons.
- Flanagan, D. (2020). *JavaScript: The Definitive Guide* (7th ed.). O'Reilly Media.
- Freeman, E., Robson, E., Sierra, K., & Bates, B. (2014). *Head First JavaScript Programming*. O'Reilly Media.
- Mozilla Developer Network. (2025). *Document Object Model (DOM)*. Retrieved from https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model
- W3Schools. (2025). *JavaScript HTML DOM*. Retrieved from https://www.w3schools.com/js/js_htmlDOM.asp
- Zulman, M. R. (2026). *Workshop web dasar: HTML, CSS, JavaScript, Express & React – Modul 6: JavaScript – DOM & events* [Modul praktikum].